MCP For Unity 使用文档

目录

- 一、介绍
- 二、使用方法
  - 2.1 编辑器使用
  - 2.2 Runtime 使用
- 三、扩展性说明
  - 3.1 编辑器扩展
  - 3.2 Runtime 扩展
- 四、附录
- 五、AI 辅助二次开发

一、介绍

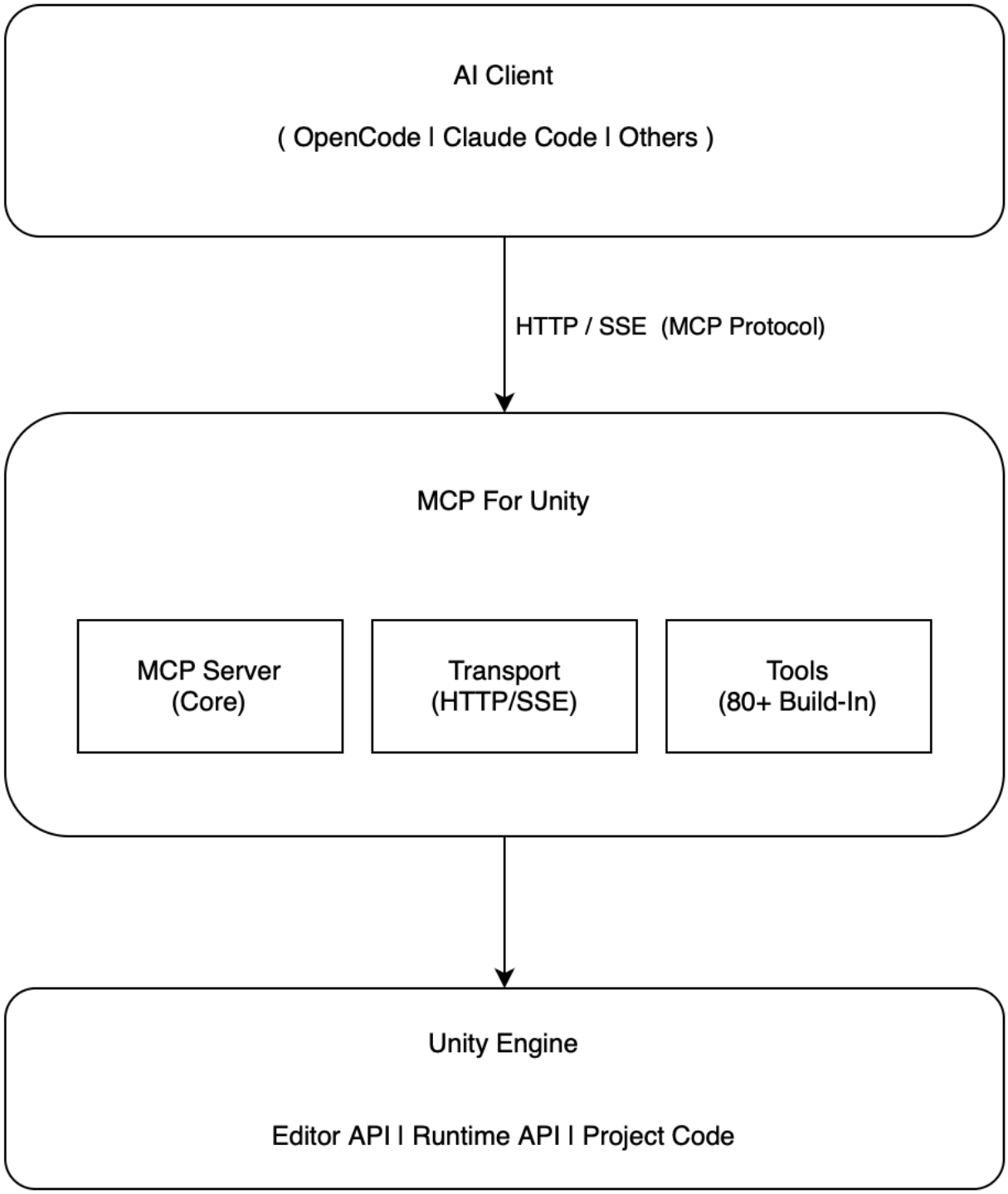
1.1 什么是 MCP For Unity

MCP For Unity 是基于 **Model Context Protocol (MCP)** 协议的 Unity 实现框架，旨在让 Unity 编辑器和运行时应用无缝融入 AI 生态系统。

核心优势：

特性	说明
纯 C# 实现	无需外部桥接、无需 Python 或其他语言依赖
完整工具链访问	AI 可直接调用 Unity 引擎 API 和项目代码
双向通信	支持 AI 与 Unity 之间的实时交互
标准化协议	兼容 Claude Desktop、VS Code MCP 等主流 AI 客户端

1.2 架构概览


AI Client
(OpenCode I Claude Code I Others)
HTTP / SSE (MCP Protocol)
MCP For Unity
MCP Server
(Core)
Transport
(HTTP/SSE)
Tools
(80+ Build-In)
Unity Engine
Editor API I Runtime API I Project Code

1.3 功能支持

功能	支持状态
Tools（工具）	✅ 完全支持
Resources（资源）	✅ 完全支持
Tasks（任务）	✅ 完全支持
Prompts（提示词）	❌ 不支持

> ⚠️ **注意**：当前版本不支持 Prompts（提示词）功能。

1.4 技术规格

项目	说明
Unity 版本	2021.3+
.NET 版本	.NET Standard 2.1
传输协议	HTTP + Server-Sent Events (SSE)
序列化	Newtonsoft.Json
依赖	`com.unity.nuget.newtonsoft-json`

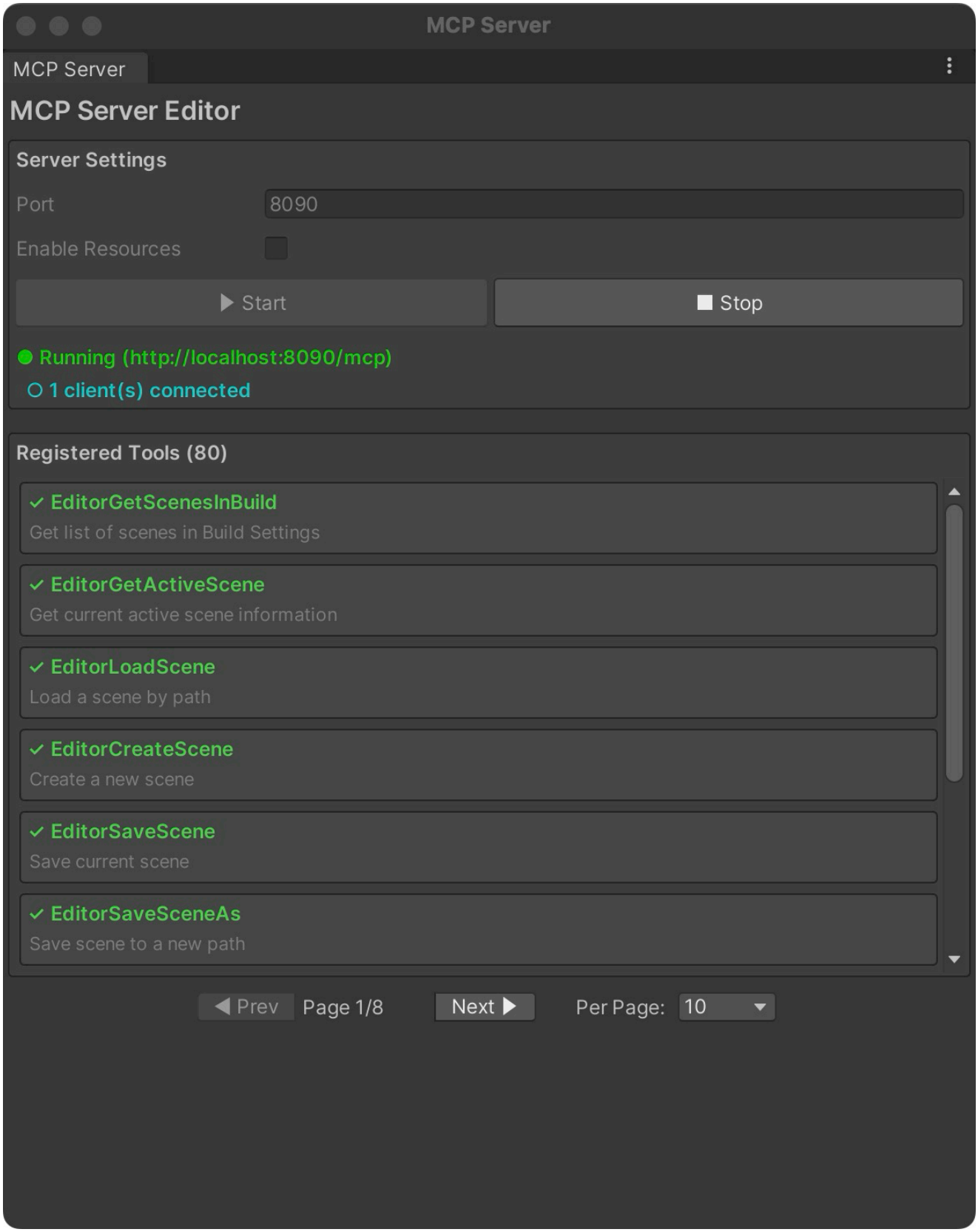
二、使用方法

2.1 编辑器使用

2.1.1 启动编辑器服务器

方式一：通过编辑器窗口

1. 打开菜单：`Tools > MCP For Unity > Server Window`
2. 配置服务器参数
3. 点击 **Start** 按钮启动


MCP Server
MCP Server
MCP Server Editor
Server Settings
Port
8090
Enable Resources
Start
Stop
Running (http://localhost:8090/mcp)
1 client(s) connected
Registered Tools (80)
EditorGetScenesInBuild
Get list of scenes in Build Settings
EditorGetActiveScene
Get current active scene information
EditorLoadScene
Load a scene by path
EditorCreateScene
Create a new scene
EditorSaveScene
Save current scene
EditorSaveSceneAs
Save scene to a new path
Prev
Page 1/8
Next
Per Page:
10

方式二：通过代码

```
using ModelContextProtocol.Editor;

// 启动（使用默认配置）
GlobalEditorMcpServer.StartServer();

// 停止
GlobalEditorMcpServer.StopServer();
```

2.1.2 配置选项

在编辑器窗口中可配置以下选项：

选项	默认值	说明
Port	8090	服务器端口
Enable Resources	false	启用资源服务
Enable File Watching	false	启用文件监视

2.1.3 内置工具说明

MCP For Unity 提供了 **80 个编辑器工具**，覆盖常见开发场景：

场景管理 (10 个)

工具名	描述
`EditorGetScenesInBuild`	获取 Build Settings 中的场景列表
`EditorGetActiveScene`	获取当前激活场景信息
`EditorLoadScene`	加载场景
`EditorCreateScene`	创建新场景
`EditorSaveScene`	保存当前场景
`EditorSaveSceneAs`	另存场景
`EditorCloseScene`	关闭场景
`EditorAddSceneToBuild`	添加场景到 Build
`EditorRemoveSceneFromBuild`	从 Build 移除场景
`EditorGetSceneHierarchy`	获取场景层级结构

GameObject 操作 (15 个)

工具名	描述
`EditorCreateGameObject`	创建空 GameObject
`EditorCreatePrimitive`	创建基础几何体
`EditorDuplicateGameObject`	复制 GameObject
`EditorDeleteGameObject`	删除 GameObject
`EditorFindGameObject`	查找 GameObject
`EditorFindGameObjectsByName`	按名称查找
`EditorFindGameObjectsByTag`	按标签查找
`EditorFindGameObjectsByLayer`	按层查找
`EditorGetGameObjectInfo`	获取详细信息
`EditorGetChildren`	获取子物体列表
`EditorSetParent`	设置父物体
`EditorSetTag`	设置标签
`EditorSetLayer`	设置层
`EditorSetStatic`	设置静态标志
`EditorSetActive`	设置激活状态

Transform 操作 (10 个)

工具名	描述
`EditorSetPosition`	设置世界位置
`EditorSetRotation`	设置世界旋转
`EditorSetScale`	设置缩放
`EditorSetLocalPosition`	设置本地位置
`EditorSetLocalRotation`	设置本地旋转
`EditorSetLocalScale`	设置本地缩放
`EditorSetPositionAndRotation`	同时设置位置旋转
`EditorTranslate`	相对移动
`EditorRotate`	相对旋转
`EditorResetTransform`	重置变换

组件操作 (10 个)

工具名	描述
`EditorAddComponent`	添加组件
`EditorRemoveComponent`	移除组件
`EditorGetComponent`	获取组件信息
`EditorGetComponents`	获取所有组件
`EditorGetComponentsInChildren`	获取子物体组件
`EditorSetComponentProperty`	设置组件属性
`EditorGetComponentProperty`	获取组件属性
`EditorHasComponent`	检查是否有组件
`EditorGetRequiredComponent`	获取或添加组件
`EditorSendComponentMessage`	发送消息

编译与构建 (10 个)

工具名	描述
`EditorGetCompilationStatus`	获取编译状态
`EditorIsCompiling`	是否正在编译
`EditorGetCompileErrors`	获取编译错误
`EditorGetCompileWarnings`	获取编译警告
`EditorGetBuildTarget`	获取构建目标
`EditorSetBuildTarget`	设置构建目标
`EditorGetScriptingBackend`	获取脚本后端
`EditorGetAssemblyDefinitions`	获取 asmdef 列表
`EditorGetDefineSymbols`	获取定义符号
`EditorSetDefineSymbols`	设置定义符号

资源管理 (15 个)

工具名	描述
`EditorFindAssets`	查找资源
`EditorGetAssetInfo`	获取资源详情
`EditorGetAssetPath`	获取资源路径
`EditorGetAssetDependencies`	获取资源依赖
`EditorGetFolderContents`	获取目录内容
`EditorCreateFolder`	创建文件夹
`EditorDeleteAsset`	删除资源
`EditorRenameAsset`	重命名资源
`EditorMoveAsset`	移动资源
`EditorCopyAsset`	复制资源
`EditorRefreshAssets`	刷新资源库
`EditorReimportAsset`	重新导入资源
`EditorGetAssetImportSettings`	获取导入设置
`EditorLoadAssetAtPath`	加载资源
`EditorGetAllAssetPaths`	获取所有资源路径

Prefab 操作 (10 个)

工具名	描述
`EditorInstantiatePrefab`	实例化预制体
`EditorCreatePrefab`	创建预制体
`EditorApplyPrefab`	应用预制体修改
`EditorRevertPrefab`	还原预制体
`EditorUnpackPrefab`	解包预制体
`EditorGetPrefabInfo`	获取预制体信息
`EditorGetPrefabType`	获取预制体类型
`EditorIsPrefabInstance`	是否是预制体实例
`EditorGetPrefabAssetPath`	获取预制体资源路径
`EditorGetAllPrefabs`	获取所有预制体

2.1.4 实现位置

```
Assets/McpForUnity/Editor/
├── Tools/
│   └── EditorToolsList.cs       # 80 个编辑器工具实现
├── GlobalEditorMcpServer.cs     # 全局服务器管理
└── Window/
    └── McpServerEditorWindow.cs # 编辑器窗口
```

2.1.5 连接 AI 客户端

Claude Desktop 配置

编辑 Claude Desktop 配置文件：

- macOS: `~/Library/Application Support/Claude/claude_desktop_config.json`
- Windows: `%APPDATA%\Claude\claude_desktop_config.json`

```
{
  "mcpServers": {
    "unity-editor": {
      "url": "http://localhost:8090/mcp",
      "transport": "sse"
    }
  }
}
```

OpenCode 配置

在项目根目录创建 `.opencode/opencode.jsonc` 文件：

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "UnityMCPServer": {
      "type": "remote",
      "url": "http://localhost:8090/mcp",
      "enabled": true
    }
  }
}
```

2.2 Runtime 使用

2.2.1 启动运行时服务器

```
using System.Threading.Tasks;
using ModelContextProtocol.Unity;
using UnityEngine;
```

```
public class MCPManager : MonoBehaviour
{
    private McpServerHost _host;

    async void Start()
    {
        // 创建配置
        var options = new McpServerHostOptions
        {
            Port = 3000,
            ServerName = "UnityMCP",
            ServerVersion = "1.0.0"
        };

        // 创建并启动服务器
        _host = new McpServerHost(options);
        await _host.StartAsync();

        // 注册自定义工具
        _host.Server.RegisterToolsFromClass(typeof(MyGameTools));

        Debug.Log($"MCP Server started at http://localhost:{options.Port}/mcp");
    }

    async void OnDestroy()
    {
        if (_host != null)
        {
            await _host.DisposeAsync();
        }
    }
}
```

> ⚠️ **重要**：Runtime 服务器**不包含任何内置工具**。所有工具需要通过 `RegisterToolsFromClass()` 或 `AddCustomTool()` 自行注册。

2.2.2 配置选项

选项	类型	默认值	说明
`Port`	int	3000	服务器端口
`ServerName`	string	"UnityMCP"	服务器名称
`ServerVersion`	string	"1.0.0"	服务器版本
`Instructions`	string	-	服务器说明
`LogLevel`	LogLevel	Information	日志级别

2.2.3 事件订阅

```
_host.OnServerStarted += () => Debug.Log("Server started");
_host.OnServerStopped += () => Debug.Log("Server stopped");
_host.OnServerError += (error) => Debug.LogError($"Server error: {error}");
```

2.2.4 注册自定义工具

```
// 方式一：通过类注册
_host.Server.RegisterToolsFromClass(typeof(MyGameTools));

// 方式二：直接添加 Lambda
_host.AddCustomTool("my_tool", "Tool description", async (args, ct) =>
{
    string param = args?["param"]?.ToString();
    // 处理逻辑
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = "Result" }
        }
    };
});
```

三、扩展性说明

3.1 编辑器扩展

3.1.1 自定义工具定义

使用属性标注定义工具：

```
using ModelContextProtocol.Protocol;
using ModelContextProtocol.Server;

public static class MyEditorTools
{
    [McpServerTool("EditorMyCustomTool", Description = "我的自定义编辑器工具")]
    public static CallToolResult MyCustomTool(
        [McpArgument(Description = "参数1", Required = true)] string param1,
        [McpArgument(Description = "参数2（可选）")] int param2 = 0)
    {
        // 执行编辑器操作
        Debug.Log($"执行自定义工具：{param1}, {param2}");

        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = $"执行成功：{param1}" }
```

```
            }
        };
    }
}
```

注册工具：

```
// 在 GlobalEditorMcpServer 启动后注册
GlobalEditorMcpServer.Server.RegisterToolsFromClass(typeof(MyEditorTools));
```

3.1.2 参数定义方法

基础类型参数

```
[McpServerTool("Example", Description = "示例")]
public static CallToolResult Example(
    [McpArgument(Description = "字符串参数", Required = true)] string text,
    [McpArgument(Description = "整数参数")] int number = 0,
    [McpArgument(Description = "浮点参数")] float value = 0f,
    [McpArgument(Description = "布尔参数")] bool flag = false)
{
    // ...
}
```

枚举参数

```
public enum MyEnum { OptionA, OptionB, OptionC }

[McpServerTool("EnumExample", Description = "枚举示例")]
public static CallToolResult EnumExample(
    [McpArgument(Description = "枚举选项")] MyEnum option = MyEnum.OptionA)
{
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"选择了: {option}" }
        }
    };
}
```

数组参数

```
[McpServerTool("ArrayExample", Description = "数组示例")]
public static CallToolResult ArrayExample(
    [McpArgument(Description = "字符串数组", Required = true)] string[] items)
{
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"收到 {items.Length} 个项目" }
        }
    };
}
```

Vector3 参数（展开形式）

```
[McpServerTool("VectorExample", Description = "Vector3示例")]
public static CallToolResult VectorExample(
    [McpArgument(Description = "位置")] UnityEngine.Vector3 position)
{
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"位置: {position}" }
        }
    };
}
```

调用时参数格式：

```
{
  "position_x": 1.0,
  "position_y": 2.0,
  "position_z": 3.0
}
```

3.1.3 自定义类型定义

定义自定义类型

```
using Newtonsoft.Json;
using ModelContextProtocol.Server;

public class PlayerData
{
    [JsonProperty("playerName")]
    [McpArgument(Description = "玩家名称", Required = true)]
    public string PlayerName;

    [JsonProperty("level")]
```

```
    [McpArgument(Description = "玩家等级")]
    public int Level;

    [JsonProperty("health")]
    [McpArgument(Description = "生命值")]
    public float Health;

    [JsonProperty("position")]
    [McpArgument(Description = "位置")]
    public UnityEngine.Vector3 Position;
}
```

在工具中使用自定义类型

```
[McpServerTool("SetPlayerData", Description = "设置玩家数据")]
public static CallToolResult SetPlayerData(
    [McpArgument(Description = "玩家数据", Required = true)] PlayerData data)
{
    Debug.Log($"玩家: {data.PlayerName}, 等级: {data.Level}");

    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"已设置玩家: {data.PlayerName}" }
        }
    };
}
```

调用格式:

```
{
  "data": {
    "playerName": "张三",
    "level": 10,
    "health": 100.0,
    "position_x": 0,
    "position_y": 1,
    "position_z": 0
  }
}
```

自定义类型验证规则

规则	说明
字段必须同时有 `[JsonProperty]` 和 `[McpArgument]`	缺少任一属性的字段将被忽略
使用 `[JsonRequired]` 或 `[McpArgument(Required = true)]`	标记必填字段
支持嵌套自定义类型	递归验证
支持自定义类型数组	`PlayerData[]` 或 `List<PlayerData>`

3.1.4 实例工具注册

实例工具允许为同一个类的多个实例注册独立的工具集。

定义实例工具类

```
[McpInstanceTool(Name = "Player", Description = "玩家实例工具")]
public class PlayerTools
{
    public string PlayerId { get; set; }
    public int Health { get; set; }
    public int Score { get; set; }

    [McpServerTool(Description = "获取玩家信息")]
    public CallToolResult GetInfo()
    {
        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = $"玩家 {PlayerId}: HP={Health}, Score=
{Score}" }
            }
        };
    }

    [McpServerTool(Description = "设置生命值")]
    public CallToolResult SetHealth(
        [McpArgument(Description = "生命值", Required = true)] int health)
    {
        Health = health;
        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = $"HP 设置为 {Health}" }
            }
        };
    }

    [McpServerTool(Description = "增加分数")]
    public void AddScore(
```

```
        [McpArgument(Description = "分数增量", Required = true)] int amount)
    {
        Score += amount;
    }
}
```

注册实例工具

```
// 创建实例
var player1 = new PlayerTools { PlayerId = "P001", Health = 100, Score = 0 };
var player2 = new PlayerTools { PlayerId = "P002", Health = 80, Score = 50 };

// 注册到服务器
GlobalEditorMcpServer.Server.RegisterToolsFromInstance(player1, "player_1");
GlobalEditorMcpServer.Server.RegisterToolsFromInstance(player2, "player_2");

// 工具名称格式：{instanceId}.{methodName}
// player_1.GetInfo
// player_1.SetHealth
// player_2.GetInfo
// player_2.SetHealth
```

卸载实例工具

```
// 卸载指定实例的所有工具
GlobalEditorMcpServer.Server.UnregisterInstanceTools("player_1");
```

注意事项

- 实例 ID 必须唯一
- 服务器持有实例引用，注意内存管理
- 不再使用时及时卸载

3.2 Runtime 扩展

Runtime 扩展方式与编辑器扩展相同。

3.2.1 自定义工具定义

```
public static class GameTools
{
    [McpServerTool("SpawnEnemy", Description = "生成敌人")]
    public static CallToolResult SpawnEnemy(
        [McpArgument(Description = "敌人类型")] string enemyType,
        [McpArgument(Description = "位置")] UnityEngine.Vector3 position)
    {
        // 游戏运行时逻辑
        var enemy = SpawnManager.SpawnEnemy(enemyType, position);
```

```
        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = $"已生成敌人: {enemy.name}" }
            }
        };
    }

    [McpServerTool("GetPlayerStatus", Description = "获取玩家状态")]
    public static CallToolResult GetPlayerStatus()
    {
        var player = GameManager.Instance.Player;
        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = JObject.FromObject(new
                {
                    health = player.Health,
                    mana = player.Mana,
                    level = player.Level
                }).ToString() }
            }
        };
    }
}
```

注册:

```
_host.Server.RegisterToolsFromClass(typeof(GameTools));
```

3.2.2 实例工具注册

```
// 定义
[McpInstanceTool(Name = "Unit", Description = "单位工具")]
public class UnitTools
{
    public Unit Unit { get; set; }

    [McpServerTool(Description = "移动到目标")]
    public CallToolResult MoveTo(
        [McpArgument(Description = "目标位置")] UnityEngine.Vector3 target)
    {
        Unit.MoveTo(target);
        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = "移动命令已发送" }
```

```
            }
        };
    }
}

// 注册
var unit = new UnitTools { Unit = myUnit };
_host.Server.RegisterToolsFromInstance(unit, "unit_001");

// 卸载
_host.Server.UnregisterInstanceTools("unit_001");
```

四、附录

4.1 McpServerHostOptions 配置表

属性	类型	默认值	说明
`Port`	int	3000	HTTP 服务端口
`ServerName`	string	"UnityMCP"	MCP 服务器名称
`ServerVersion`	string	"1.0.0"	服务器版本
`Instructions`	string	-	发送给 AI 的使用说明
`LogLevel`	LogLevel	Information	日志输出级别

4.2 事件列表

事件	签名	触发时机
`OnServerStarted`	`Action`	服务器启动成功
`OnServerStopped`	`Action`	服务器停止
`OnServerError`	`Action<string>`	服务器发生错误

4.3 返回结果类型

成功结果

```
return new CallToolResult
{
    Content = new List<ContentBlock>
    {
        new TextContentBlock { Text = "操作成功" }
    }
};
```

错误结果

```
return new CallToolResult
{
    IsError = true,
    Content = new List<ContentBlock>
    {
        new TextContentBlock { Text = "错误信息" }
    }
};
```

带图片的结果

```
return new CallToolResult
{
    Content = new List<ContentBlock>
    {
        new ImageContentBlock
        {
            Data = base64ImageData,
            MimeType = "image/png"
        }
    }
};
```

4.4 注意事项

1. **线程安全**：MCP 工具在后台线程执行，操作 Unity 对象时需使用 `MainThreadDispatcher` 或确保在主线程执行
2. **生命周期**：`McpServerHost` 会自动监听 `Application.quitting`，应用退出时自动清理
3. **安全性**：默认监听 `localhost`，不接受外部连接。生产环境需谨慎配置
4. **性能**：大量工具调用可能影响性能，建议合理控制调用频率
5. **错误处理**：工具内部异常会被捕获并返回错误结果，不会导致服务器崩溃

4.5 文件结构

```
Assets/McpForUnity/
├── Core/
│   ├── IMcpServerHost.cs          # 服务器接口
│   ├── McpServerHost.cs           # 服务器实现
│   ├── McpServerHostOptions.cs    # 配置类
│   ├── McpException.cs            # 异常类型
│   ├── Throw.cs                   # 参数验证
│   └── Protocol/                  # MCP 协议类型
├── Server/
│   ├── McpServer.cs               # 核心服务器
│   ├── McpServerOptions.cs        # 服务器配置
│   └── Tools/
```

```
│       └── Attributes.cs          # 工具属性定义
├── Transport/
│   ├── ITransport.cs              # 传输接口
│   └── HttpListenerServerTransport.cs
├── Editor/
│   ├── Tools/
│   │   └── EditorToolsList.cs     # 80 个编辑器工具
│   ├── GlobalEditorMcpServer.cs   # 编辑器服务器
│   └── Window/
│       └── McpServerEditorWindow.cs
├── Utilities/
│   ├── UnityLogger.cs             # 日志实现
│   └── MainThreadDispatcher.cs    # 主线程调度
└── Samples/
    └── MCPExampleUsage.cs         # 使用示例
```

4.6 常见问题

Q: 端口被占用怎么办?

A: 修改 `Port` 配置为其他端口，或释放占用的端口。

Q: Newtonsoft.Json 未找到?

A: 确保已安装 `com.unity.nuget.newtonsoft-json` 包:

```
Window > Package Manager > + > Add package by name
输入: com.unity.nuget.newtonsoft-json
```

Q: 如何在工具中访问 MonoBehaviour?

A: 使用 `GameObject.Find()` 或单例模式获取场景中的对象引用。

Q: 工具参数如何传递复杂对象?

A: 定义自定义类型，使用 `[JsonProperty]` 和 `[McpArgument]` 标注字段。

五、AI 辅助二次开发

MCP For Unity 支持通过 AI 客户端进行二次开发。只需将项目配置文件复制到工作目录，即可让 AI 助手理解项目结构并协助开发。

5.1 快速开始

步骤一：复制配置文件

将 `AGENTS.md` 复制到项目根目录:

```
# macOS / Linux
cp Assets/McpForUnity/AGENTS.md ./

# Windows
copy Assets\McpForUnity\AGENTS.md .
```

步骤二：启动 AI 客户端

使用支持 MCP 协议的 AI 客户端（如 OpenCode、Claude Desktop 等）打开项目目录。

步骤三：开始开发

AI 助手将自动读取 `AGENTS.md` 中的项目信息，包括：

- 代码规范和命名约定
- 项目结构和文件位置
- 构建和测试命令
- API 使用方法

5.2 推荐工具

工具	说明
OpenCode	推荐使用，原生支持 MCP 协议
Claude Desktop	需配置 MCP 服务器
Cursor	支持 MCP 扩展
VS Code + MCP 插件	需安装 MCP 扩展

5.3 开发流程

```
1. AI 读取 AGENTS.md → 理解项目规范
2. 启动 MCP Server → AI 可调用 Unity 工具
3. 描述需求 → AI 生成/修改代码
4. AI 调用工具 → 验证/测试代码
5. 迭代优化 → 完成开发
```

5.4 示例场景

场景	示例提示
添加新工具	"添加一个导出场景层级到 JSON 的编辑器工具"
修复问题	"EditorInstantiatePrefab 工具报错，帮我修复"
代码重构	"按照项目规范重构这个工具类"
编写测试	"为 EditorCreateGameObject 编写单元测试"
添加文档	"为新工具添加中文注释和使用说明"

5.5 注意事项

- AI 助手通过 MCP 协议可以直接操作 Unity 编辑器
- 建议在 AI 修改代码后进行编译检查
- 复杂操作建议分步骤进行，便于验证和回滚

文档版本: 1.0.3
最后更新: 2026-03-02